

CMSC828G: Special Topics in Systems for Machine Learning

Compilers and MLIR

Instructor: Yafan Huang



DEPARTMENT OF
COMPUTER SCIENCE

From Hand-Written Kernels to Compiler-Generated Kernels

Original GPU Kernel
(CUDA / Trinton)



Efficient GPU Kernel
(CUDA / Trinton)

*Tiling, memory
coalescing, shared
memory, tensor
cores, warp
specification, ...*

Pytorch / Tensor
Program

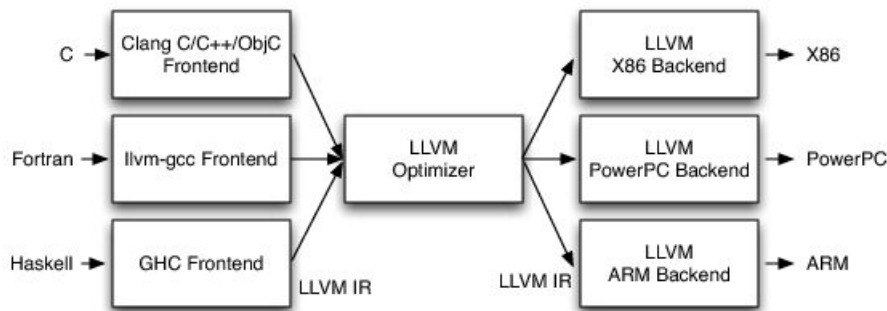


Efficient GPU Kernel
(CUDA / Trinton)



Compiler = Representation + Transformation

- Intermediate Representation (IR): A program (or source code) level that makes certain properties explicit.
- Pass: An analysis or transformation over an IR.



*LLVM Compiler
Infrastructure*

- Compiler optimization is largely about choosing the right representation for the transformation you want to perform.

Example of Transformation Passes

- Data-flow optimization: lazy code motion, dead code elimination, ...
- Control-flow optimization: loop unrolling, loop fusion, loop vectorization, ...
- Memory access optimizations: mem2reg, tail call elimination, ...

```
// original loop
int x;
for (x = 0; x < 100; x++)
{
    delete(x);
}

// Loop after unrolling (stride size = 4)
int x;
for (x = 0; x < 100; x += 5 )
{
    delete(x);
    delete(x + 1);
    delete(x + 2);
    delete(x + 3);
    delete(x + 4);
}
```

Loop Unrolling

```
// original loop
for (int i = 0; i < n; i++) {
    a[i] = b[i] + c[i];
}

// Loop after vectorization
for (int i = 0; i < n; i += 4) {
    __m128d b_vec = _mm_load_pd(&b[i]);
    __m128d c_vec = _mm_load_pd(&c[i]);
    __m128d sum_vec = _mm_add_pd(b_vec, c_vec);
    _mm_store_pd(&a[i], sum_vec);
}
```

Loop Vectorization

```
// fused loop
int i, a[100], b[100];
for (i = 0; i < 100; i++) {
    a[i] = 1;
    b[i] = 2;
}

// ↑ to ↓: Loop fission
// ↓ to ↑: Loop fusion

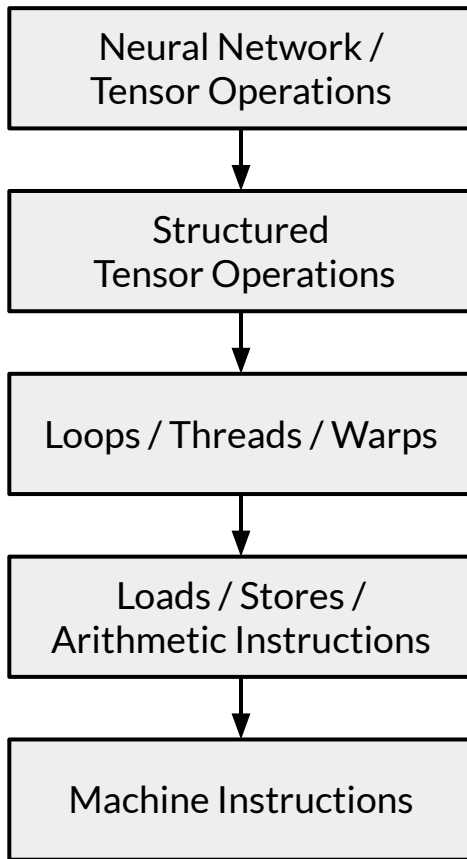
// fissioned loop
int i, a[100], b[100];
for (i = 0; i < 100; i++) {
    a[i] = 1;
}
for (i = 0; i < 100; i++) {
    b[i] = 2;
}
```

Loop Fusion / Fission

Is One IR Enough?

$$Y = \text{ReLU}(XW + B)$$

Compile
from
High-level
to Low-level



MatMul, Bias, ReLU, Tensor shapes

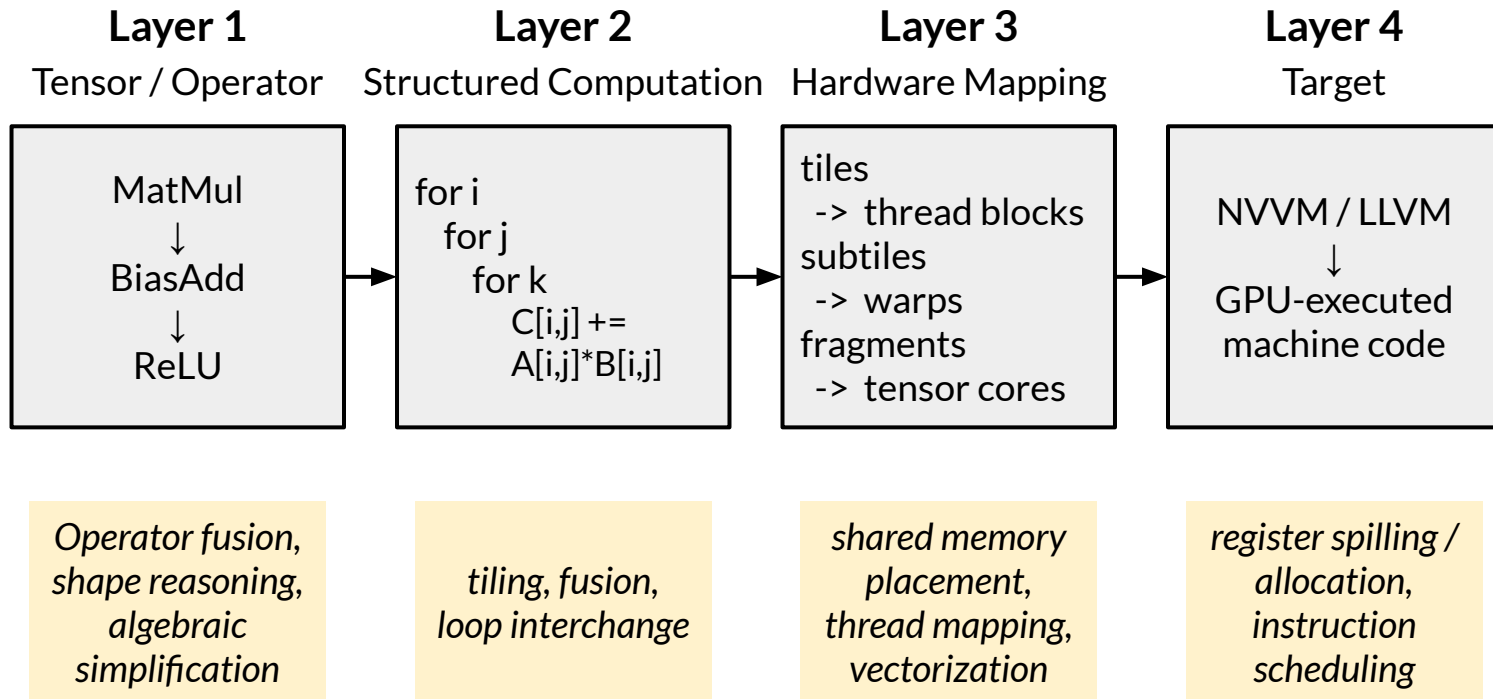
load, load, fmul, branch, store, ...

MLIR: Multi-Level Intermediate Representation

- MLIR is an intermediate representation (IR) framework and tries to preserve the right abstraction at the right time.
- A dialect defines operations, types, and semantics for a particular abstraction level or domain within that IR.
 - Tensor dialect, loop dialect, GPU dialect, LLVM / NVVM dialect, ...

Multiple Levels of Representation for One Computation

$$Y = \text{ReLU}(XW + B)$$



A Lens for Today's Papers

- What abstraction level does this compiler operate on?
 - Graph? Tensor? Loop? Kernel? Instruction?
- What optimization decisions does it automate?
 - Fusion? Tiling? Layout? Scheduling? Parallelization? Code generation?
- What is the tradeoff?
 - Performance? Portability? Productivity (or review efforts)?
 - Compilation cost? Generality?

Student Presentations

- [OSDI'25] KPerfIR: Towards an Open and Compiler-centric Ecosystem for GPU Kernel Performance Tooling on Modern AI Workloads
- [MLSys'26] ApproxMLIR: Accuracy-Aware Compiler for Compound ML System